# Procesamiento de Lenguajes (PL)
# Curso 2025/2026

## Práctica 1: analizador descendente recursivo

### Fecha y método de entrega

La práctica debe realizarse de forma individual, y debe entregarse a través del servidor de prácticas del DLSI **antes de las 23:59 del 6 de marzo de 2026**. Los ficheros fuente en Java se comprimirán en un fichero llamado "`plp1.tgz`", y no debe haber directorios (ni `package`) en él, solamente los fuentes en Java.

Al servidor de prácticas del DLSI se puede acceder desde la web del DLSI (http://www.dlsi.ua.es), en el apartado "Entrega de prácticas". Una vez en el servidor, se debe elegir la asignatura PL y seguir las instrucciones.

### Descripción de la práctica

La práctica consiste en implementar un analizador sintáctico descendente recursivo, como se ha estudiado en la clase de teoría, utilizando Java como lenguaje fuente.

Como subprograma del analizador sintático será necesario realizar un analizador léxico, que será utilizado en la práctica 2. Además, el analizador léxico y sintáctico serán la base de la práctica 3.

### Analizador léxico

El analizador léxico debe reconocer los siguientes componentes léxicos:

| Token | Expresión regular | Cadena a mostrar |
|---|---|---|
| pari | `(` | `(` |
| pard | `)` | `)` |
| dosp | `:` | `:` |
| lbra | `{` | `{` |
| rbra | `}` | `}` |
| asig | `=` | `=` |
| pyc | `;` | `;` |
| oprel | `(([<>=!]=)\|[<>])` | `< <= > >= == !=` |
| opas | `[-+]` | `+ -` |
| opmul | `[*/]` | `* /` |
| class | `class` | `'class'` |
| fun | `fun` | `'fun'` |
| int | `int` | `'int'` |
| float | `float` | `'float'` |
| if | `if` | `'if'` |
| else | `else` | `'else'` |
| fi | `fi` | `'fi'` |
| print | `print` | `'print'` |
| id | `[a-zA-Z][a-zA-Z0-9]*` | `identificador` |
| numentero | `[0-9]+` | `numero entero` |
| numreal | `[0-9]+\.[0-9]+` | `numero real` |

Ten en cuenta las siguientes cuestiones:

1. Debes diseñar el diagrama de transiciones en papel, tal y como se ha estudiado en clase de teoría. Para esta práctica no es necesario que pongas las palabras reservadas en el diagrama (saldrían muchos estados), puedes tratar las palabras reservadas como identificadores en el diagrama, y justo antes de devolver el token, comprobar

si el identificador coincide con alguna de las palabras reservadas para devolver el token asociado en vez del token del identificador.

2. El lenguaje es sensible a mayúsculas, es decir, "`print`" es el token `print` y "`prinT`" es un identificador.

3. El analizador léxico debe ignorar los espacios en blanco, tabuladores y saltos de línea (excepto para contar columnas y filas, en cuyo caso el tabulador se contará como un único caracter). Debe leer el fichero carácter a carácter, hasta completar un token. En ningún caso debe leer el fichero completo en memoria, ni leer más de una vez el fichero.

4. En caso de que algún carácter de la entrada no pueda formar parte de un token, se debe emitir el siguiente error:

   ```
   Error lexico (f,c): caracter 'a' incorrecto
   ```

   donde `f` es la fila, `c` es la columna, y `a` es el carácter incorrecto. Cuando se encuentre un error, el programa debe terminar con `System.exit(-1)`

   Si el carácter incorrecto es el que representa el final del fichero, entonces el mensaje debe ser:

   ```
   Error lexico: fin de fichero inesperado
   ```

5. El analizador léxico también debe ignorar los comentarios, que comenzarán por "`/*`" y se extenderán (posiblemente a lo largo de varias líneas) hasta que aparezca la secuencia "`*/`". No se permiten comentarios anidados, es decir, el primer "`*/`" cierra el primer "`/*`". Si se termina el fichero en mitad de un comentario se debe emitir el error de fin de fichero inesperado.

6. En los programas que se utilizarán para probar las prácticas solamente habrá caracteres del código ASCII, es decir, no habrá acentos ni `ñ` ni otros caracteres extraños.

7. El analizador léxico se debe implementar con dos clases en Java:

   - La clase `Token`, que tendrá los tipos de token de la especificación, tal y como se ha explicado en la clase de teoría. El método `toString` debe devolver para cada token la cadena que aparece en la tercera columna de la tabla anterior.
   - La clase `AnalizadorLexico`, que debe implementar al menos el método público `siguienteToken`, que leerá caracteres de la entrada hasta formar un token y lo devolverá.

8. Utiliza la clase `RandomAccessFile` para acceder al fichero de entrada, usando el método `readByte()` o `read()` para leer caracteres. El método `readChar()` lee caracteres Unicode y no sirve para leer caracteres ASCII.

## Analizador sintáctico

La gramática que debe utilizarse como base para diseñar el analizador sintáctico es la siguiente:

$$
\begin{array}{lcl}
S & \longrightarrow & \textbf{class id lbra } M \textbf{ rbra} \\
M & \longrightarrow & Fun\ M \\
M & \longrightarrow & S\ M \\
M & \longrightarrow & \epsilon \\
Fun & \longrightarrow & \textbf{fun id } A \textbf{ lbra } M\ Cod \textbf{ rbra} \\
A & \longrightarrow & DV\ Ap \\
Ap & \longrightarrow & \textbf{pyc}\ DV\ Ap \\
Ap & \longrightarrow & \epsilon \\
DV & \longrightarrow & Tipo \textbf{ id} \\
Tipo & \longrightarrow & \textbf{int} \\
Tipo & \longrightarrow & \textbf{float} \\
Cod & \longrightarrow & I\ Codp \\
Codp & \longrightarrow & \textbf{pyc}\ I\ Codp \\
Codp & \longrightarrow & \epsilon \\
I & \longrightarrow & DV \\
I & \longrightarrow & \textbf{lbra } Cod \textbf{ rbra} \\
I & \longrightarrow & \textbf{id asig } Expr \\
I & \longrightarrow & \textbf{if } Expr \textbf{ dosp } I\ Ip \\
Ip & \longrightarrow & \textbf{else } I \textbf{ fi} \\
Ip & \longrightarrow & \textbf{fi} \\
I & \longrightarrow & \textbf{print } Expr \\
Expr & \longrightarrow & E\ Exprp \\
Exprp & \longrightarrow & \textbf{oprel } E \\
Exprp & \longrightarrow & \epsilon \\
E & \longrightarrow & T\ Ep \\
Ep & \longrightarrow & \textbf{opas } T\ Ep \\
Ep & \longrightarrow & \epsilon \\
T & \longrightarrow & F\ Tp \\
Tp & \longrightarrow & \textbf{opmul } F\ Tp \\
Tp & \longrightarrow & \epsilon \\
F & \longrightarrow & \textbf{id} \\
F & \longrightarrow & \textbf{numentero} \\
F & \longrightarrow & \textbf{numreal} \\
F & \longrightarrow & \textbf{pari } Expr \textbf{ pard}
\end{array}
$$

pyc -> punto y coma

DV ->declaración variable

Ten en cuenta las siguientes cuestiones:

1. Para probar el analizador sintáctico necesitas el analizador léxico de la práctica funcionando correctamente.
2. Calcula los conjuntos de predicción y comprueba que la gramática resultante es LL(1). Como puedes comprobar, ya se ha eliminado la recursividad por la izquierda y los factores comunes por la izquierda de la gramática.
3. En caso de que el analizador encuentre un error, se debe emitir el siguiente mensaje:

   ```
   Error sintactico (f,c): encontrado 'token', esperaba ...
   ```

   donde `f` es la fila, `c` es la columna (del primer carácter del token), y `token` es el lexema del token incorrecto, y después de `esperaba` debe aparecer una lista con los tokens que se esperaba, mostrando para cada token la cadena que aparece en la tercera columna de la tabla de los componentes léxicos. Como en el analizador léxico, si aparece algún error se debe terminar la ejecución con `System.exit(-1)`

**Ejemplo:**

```
Error sintactico (4,5): encontrado 'hola', esperaba ; 'fi' } ) + -
```

Los tokens esperados deben aparecer en este orden (el que tienen en la gramática): **class**, **id**, **lbra**, **rbra**, **fun**, **pyc**, **int**, **float**, **asig**, **if**, **dosp**, **else**, **fi**, **print**, **oprel**, **opas**, **opmul**, **numentero**, **numreal**, **pari**, **pard**.

Si el token encontrado es el final del fichero, el mensaje de error debe ser:

```
Error sintactico: encontrado fin de fichero, esperaba ...
```

4. El analizador sintáctico se debe implementar en una clase en Java, denominada `AnalizadorSintacticoDR`, que tendrá al menos los métodos/funciones asociados a los no terminales de la gramática.

5. Se publicará un script para comprobar que el analizador sintáctico funciona razonablemente bien. Se recomienda utilizar un método privado para acumular los números de regla, y un atributo booleano (flag) en la parte privada de la clase que permita mostrar o no los números de las reglas que ha ido aplicando el analizador sintáctico, de forma que se pueda reutilizar fácilmente el código para la práctica 3. Por ejemplo, ante este fichero de entrada (y suponiendo que el atributo booleano está a `true`):

```
class Main {
  fun main  int a {

    class A {
    }

    /* codigo main */

    print 1+2-(3.4)
  }
}
```

la salida del analizador debería ser:[1]

```
 1 2 5 6 9 10 8 3 1 4 4 12 21 22 25 28 32 30 26 28 32 30 26 28 34 22 25 28 33 30 27 24 30 27 24 14 4
```

6. Los números de regla se deben ir acumulando (utilizando un atributo privado de tipo `StringBuilder`, y un método privado), y cuando el análisis termine con éxito, en el método `.comprobarFinFichero()`, se imprimirán los números de regla. Si se produce algún error léxico o sintáctico, no saldrá ningún número de regla.

7. La práctica debe tener varias clases en Java:

   - La clase `plp1`, que tendrá solamente el siguiente programa principal (y los `import` necesarios, sin declarar un `package`):

```
class plp1 {
    public static void main(String[] args) {

        if (args.length == 1)
        {
          try {
            RandomAccessFile entrada = new RandomAccessFile(args[0],"r");
            AnalizadorLexico al = new AnalizadorLexico(entrada);
            AnalizadorSintacticoDR asdr = new AnalizadorSintacticoDR(al);

            asdr.S(); // simbolo inicial de la gramatica
            asdr.comprobarFinFichero();
          }
          catch (FileNotFoundException e) {
            System.out.println("Error, fichero no encontrado: " + args[0]);
          }
        }
        else System.out.println("Error, uso: java plp1 <nomfichero>");
    }
}
```

[1] En una única línea.

- La clase `AnalizadorSintacticoDR`, que tendrá los métodos/funciones asociados a los no terminales del analizador sintáctico, que imprimirán las reglas que van aplicando (según el valor del atributo booleano mencionado anteriormente).
- Las clases `Token` y `AnalizadorLexico` del analizador léxico